

Fonctions:

```
1 copy(grid)
2 compareGrid(grid1, grid2)
3 createHintList(row)
4 printGrid(grid)
5 allPossibilityList(hintList, possibilityLength)
6 reducePossibilities(rowNb)
7 editGrid(rowNb)
8 solve()
9 finPartie()
```

Variables:

```
var gridSolution = [ ]
var grid = [ ]
row = [ ]
var columnSolution = [ ]
column = [ ]
var hintListRows = [ ]
var hintListColumns = [ ]
var allPossibility = [ ]
res = [ ]
hintList = [ ]
emptyList = [ ]
possibility = [ ]
rowNb
gridHeight = 10
gridWidth = 10
```

1. gridSolution = []

On commence par créer une liste gridSolution et on la remplit avec 1 ou 2 d'une manière aléatoire avec l'aide des fonctions suivantes:

Math.random() * 2

- Renvoie un nombre décimal aléatoire compris entre 0 inclus et 1 exclu
- Puis on multiplie par 2 → le nombre est maintenant entre 0 et 2 (toujours 2 exclu)

Math.floor()

- Arrondie les nombres aléatoires vers le bas
 - entre 0 et <1 → 0
 - entre 1 et <2 → 1

Et à la fin on ajoute +1, donc $0 + 1 = 1$ et $1 + 1 = 2$!

```

gridSolution
  Col 0  Col 1  Col 2  Col 3
  ↓     ↓     ↓     ↓
L0 → [1]  [0]  [1]  [0]
L1 → [0]  [1]  [1]  [0]
L2 → [1]  [1]  [0]  [0]

```

```
row = [ ]
```

La liste gridSolution consiste de plusieurs listes appelés row. Ceux sont ces listes là qui sont utilisées pour la fonction createHintList (cf 5.createHintList plus bas)

2. grid = []

Initialisation de la grille de jeu (celle que nous allons utiliser pour jouer.)

On prends le gridHeight et le gridWidth, on génère les listes de lignes (row) et on les remplit de cases de jeu vides. A la fin on ajoute tous les lignes générés lors de la boucle à la liste “grid”.

Maintenant le grid est une liste de listes de lignes.

```

grid = [
  [0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0],
  [0, 0, 0, 0, 0]
]

```

3 columnsSolution

fait la même chose que gridSolution, mais avec les colonnes:

```

Colonne 0 =
  [ gridSolution[0][0],
    gridSolution[1][0],
    gridSolution[2][0] ]
  → [1, 0, 1]

```

```

columnsSolution = [
  [1, 0, 1], // colonne 0
  [0, 1, 1], // colonne 1
  [1, 1, 0], // colonne 2
  [0, 0, 0] // colonne 3
]

```

4 En se basant sur ces deux listes (gridSolution et columnsSolutions) on va créer les:

hintListRows et hintListColumns en utilisant la fonction createHintList.

5 CreateHintList:

on initie le hintList = [] et indice = 0

puis on va parcourir les listes (listes à l’interieur de gridSolution et columnsSolution) et si la valeur d’indice est égale à 1 on va incrémenter l’indice. Chaque boucle va également vérifier qu’il ne s’agit pas de la dernière case de la liste, si c’est le cas et la valeur est égale à 1 il va ajouter la valeur de l’indice à la liste HintList et arrêter la boucle de cette ligne.

Si la prochaine valeur est égale à 0, on va d’abord ajouter la valeur d’indice à la liste “hintList” et puis la réinitialiser à 0.

Uniquement les valeurs supérieures à 0 sont ajoutés à la liste des indices hintList.

Si aucune case de la liste n'est rempli par 1, et la liste hintList demeure vide, on va ajouter un 0 à la hintList.

Ligne : 

Indices: 2 1 3

Donc pour générer les listes hintListRows on va parcourir la liste des listes « gridSolution » en utilisant la fonction createHintList et créer une liste des listes d'indices. (cf. exemple visuel ci-dessous)

Et hintListColumns on va parcourir la liste des listes « columnSolution » et créer une liste des listes d'indices.

Exemple visuel :

<pre>gridSolution = [[1,1,0,0,1,1,1,0,0,1], [0,0,0,1,1,0,0,1,1,1], [1,1,1,0,0,0,1,1,0,0], [0,1,1,1,0,1,1,0,0,0], [1,0,0,0,1,1,1,1,0,0], [1,1,0,1,0,0,0,1,1,1], [0,0,1,1,1,0,1,0,0,1], [1,1,0,0,1,1,0,0,1,1], [0,1,1,1,0,0,1,1,1,0], [1,0,0,1,1,0,0,0,1,1]]</pre>	 createHintList(gridSolution[i]) 	<pre>hintListRows = [[2,3,1], [2,3], [3,2], [3,2], [1,4], [2,1,3], [3,1,1], [2,2,2], [3,3], [1,2,2]]</pre>
--	---	--

Puis on va utiliser la fonction allPosibilityList pour générer l'allPossibility = []

6 allPossibilityList(hintList, possibilityLength)

possibilityLength = la taille de la Ligne ou colonne associé à hintlist.

On commence par initialiser une liste res = []. Cette liste va contenir toutes les possibilités de combinaisons d'une ligne.

Si la valeur de la hintList est égale à 0, on va initier une nouvelle liste possibility = [] et on va le remplir de 2 (la valeur utilisé pour les cases non remplies lors de la remplissage aléatoire). Puis on va ajouter cette liste à la liste « res ». Cette partie n'est peut-être pas nécessaire pour le bon fonctionnement du code.

Puis on va initier :

emptyList = [] → tailles des espaces vides

sommeHintList = 0 → total des cases remplies

sommeEmptyList = 0 → total des cases vides obligatoires

On commence par la possibilité qu'il n'y a pas d'espaces vides entre première indice de la hintList[0] et ajoutons 0 à la emptyList :

if (i == 0) emptyList.push(0)

et si l'indice n'est pas égale à 0, on ajoute 1 à la emptyList.
et puis on ajoute la valeur de l'indice de hintList à la sommeHintList et la valeur de la liste emptyList à la sommeEmptyList.

Exemple :

```
hintList = [3, 2]    // deux groupes : 3 cases puis 2 cases  
possibilityLength = 10
```

1ere boucle :

```
emptyList.push(0)  
sommeHintList += 3  
sommeEmptyList += 0  (nous avons dit qu'il n'y a pas d'espaces vides avant la première indice)
```

2eme boucle :

```
emptyList.push(1)  
sommeHintList += 2  
sommeEmptyList += 1  (nous avons dit qu'il y a minimum 1 case vide entre chaque plage remplie)
```

A la fin de la boucle :

```
emptyList = [0, 1]  
sommeHintList = 5  
sommeEmptyList = 1
```

puis on calcule leftover = possibilityLength - sommeHintList – sommeEmptyList pour déterminer le nombre de cases vides restantes et ajoutons cette valeur à la emptyList.

EmptyList = [0, 1, 4] :

0 vide avant le 1er groupe
1 vide entre les groupes
4 vides après le dernier groupe

Cette structure te permet ensuite de :

- déplacer les groupes vers la droite
- générer toutes les possibilités valides
- comparer chaque possibilité avec la grille du joueur

Puis on lance une boucle while

La boucle while génère toutes les configurations possibles d'une ligne en fonction des blocs (hintList) et des espaces disponibles (emptyList).

À chaque itération, elle construit une possibilité complète à partir de la répartition actuelle des espaces, l'ajoute à la liste des résultats, puis dans la deuxième partie elle modifie emptyList pour préparer la configuration suivante.

Elle se répète jusqu'à ce que tous les espaces aient été progressivement déplacés vers la gauche, ce qui signifie que toutes les combinaisons ont été générées.

1 La boucle while commence.

2 On construit une ligne complète (possibility) à partir de la emptyList actuelle.

3 On ajoute cette ligne dans res.

4 On exécute la boucle for qui modifie emptyList pour obtenir la configuration suivante.

5 Dès qu'une modification est faite, le break arrête la boucle for.

6 Le while recommence alors du début avec la nouvelle emptyList.

Initialisation d'une liste possibility = []

La première boucle parcourt emptyList et utilise le même indice pour accéder à hintList.

Pour chaque espace :

1. Une boucle interne ajoute autant de croix (valeur 2) que le nombre indiqué dans emptyList[i].
2. Si cet espace n'est pas le dernier, une seconde boucle interne ajoute un bloc de cases remplies (valeur 1), dont la taille correspond à hintList[i].

Exemple :

hintList = [3, 2]

possibilityLength = 10

emptyList = [0, 1, 4]

Structure logique :

$E0 + \text{bloc}(3) + E1 + \text{bloc}(2) + E2$

($E0 = 0$, $E1 = 1$, $E2 = 4$)

Déroulement :

- $i = 0$
 $E0 = 0 \rightarrow$ aucune croix ajoutée
 $\text{bloc}(3) \rightarrow$ ajout de trois 1
 $\text{possibility} = [1, 1, 1]$
- $i = 1$
 $E1 = 1 \rightarrow$ ajout d'un 2
 $\text{bloc}(2) \rightarrow$ ajout de deux 1
 $\text{possibility} = [1, 1, 1, 2, 1, 1]$
- $i = 2$
 $E2 = 4 \rightarrow$ ajout de quatre 2
Comme il s'agit du dernier espace, aucun bloc n'est ajouté.

(if ($i \neq \text{emptyList.length} - 1$) est fausse ($i=2 \rightarrow \text{emptyList.length} = 3$ et on soustrait 1) on ne rentre plus dans cette boucle)

Résultat final : $\text{possibility} = [1, 1, 1, 2, 1, 1, 2, 2, 2, 2]$

Après chaque construction, la possibilité est ajoutée à la liste res.

Boucle while se termine une fois que la première indice de la liste emptyList est égale au leftover.

Cela signifie que tout l'espace libre a été déplacé vers la gauche. Donc toutes les configurations possibles ont été générées.

Visualisation de la boucle while et la liste res.

E0	E1	E2	Ligne générée (liste res)
0	1	4	111 2 11 2222
1	1	3	2 111 2 11 222
2	1	2	22 111 2 11 22
3	1	1	222 111 2 11 2
4	1	0	2222 111 2 11

La partie suivante avec boucle `for (let i = emptyList.length-1; i >= 0; i--)` (ligne 173)

sert à générer la configuration suivante en modifiant la répartition des espaces (emptyList) et déplacer progressivement les croix vers la gauche. Après chaque nouvelle génération de emptyList

On commence en parcourant emptyList de droite vers la gauche.

On cherche : Règle :

- Le dernier espace peut descendre jusqu'à 0
- Les espaces du milieu doivent rester au minimum à 1
- Le premier peut aller jusqu'au maximum (leftover)

Cas 1 :

if (i == emptyList.length-1) => on regarde le dernier espace

emptyList[i] > 0 => si la valeur est supérieure à 0 alors

emptyList[i] -= 1 => on prend un croix du dernier espace

emptyList[i-1] += 1 => et on l'ajoute à l'espace de gauche.

Exemple :

emptyList = [0, 1, 4]

i = 2 (dernier espace)

la valeur de i : 4 > 0 → condition vraie, donc on applique :

emptyList[1] += 1 → 1 devient 2

emptyList[2] -= 1 → 4 devient 3

Après modification : emptyList = [0, 2, 3]

Cas 2 :

if (emptyList[i] > 1) => on regarde un espace du milieu

on regarde >1 parce qu'un espace central doit toujours garder **au moins 1 croix** pour séparer les blocs.

emptyList[i-1] += 1 => on ajoute un croix à gauche

emptyList[i] -= 1 => on enleve un à l'espace de milieu

$\text{emptyList}[\text{emptyList.length}-1] = \text{emptyList}[i] - 1 \Rightarrow$ On redistribue le surplus complètement à droite

$\text{emptyList}[i] = 1 \Rightarrow$ On remet l'espace courant au minimum (1) (puisque il faut garder au moins 1 entre les blocks)

Exemple:

$\text{emptyList} = [0, 3, 1]$

on regarde l'espace du milieu: $i = 1$

la valeur est $3 > 1 \rightarrow$ condition vraie, donc on applique:

$\text{emptyList}[0] += 1 \rightarrow 0$ devient 1

$\text{emptyList}[1] -= 1 \rightarrow 3$ devient 2

$\text{emptyList}[\text{emptyList.length}-1] = \text{emptyList}[1] - 1 \Rightarrow \text{emptyList}[2] = 2 - 1 = 1$

$\text{emptyList}[1] = 1$

Résultat final: $\text{emptyList} = [1, 1, 1]$

Break intervient après chaque boucle et la nouvelle `emptyList` est utilisé pour générer une nouvelle liste `possibility` qui est ensuite enregistré dans la liste `res`.

A la fin la fonction `allPossibilityList` va retourner une liste `res` avec toutes les possibilités.

`reducePossibilities(rowNb)`

Cette fonction sert à :

Supprimer les possibilités qui ne correspondent pas à l'état actuel de la grille.

Elle fonctionne pour :

- une ligne (si $\text{rowNb} < \text{gridHeight}$)
- une colonne (si $\text{rowNb} \geq \text{gridHeight}$)

Pour la ligne ou colonne numéro `rowNb` :

1. On parcourt toutes ses possibilités.
2. Pour chaque possibilité, on vérifie chaque case.
3. Si une case contredit la grille, on supprime la possibilité.

Pour cette fonction nous utilisons la liste `allPossibilities` $\Rightarrow$ C'est un tableau qui contient toutes les possibilités de toutes les lignes et toutes les colonnes. Pour avoir cette liste nous faisons appel à la fonction `allPosibilityList`.

`allPossibility[0]` $\rightarrow$ toutes les possibilités de la ligne 0

`allPossibility[1]` $\rightarrow$ toutes les possibilités de la ligne 1 etc.

et idem pour les colonne :

`allPossibility[gridHeight] → colonne 0`

`allPossibility[gridHeight + 1] → colonne 1`

Et nous allons comparer cette liste avec la liste grid qui est la grille de jeu (celle que nous allons utiliser pour jouer.)

`while (i < allPossibility[rowNb].length) =>` On parcourt toutes les possibilités de cette ligne/colonne. On utilise un `while` au lieu d'un `for` parce qu'on supprime des éléments avec `splice`.

`for (let j = 0; j < allPossibility[rowNb][i].length; j++) =>` On compare chaque cellule de la possibilité avec la grille.

Puis la condition : si la possibilité est différente de la grille ET que la grille n'est pas inconnue ($\neq 0$)
→ alors cette possibilité est invalide et est supprimé de la liste des possibilités.

Le `grid != 0` empêche d'éliminer des possibilités à cause de cases encore indéterminées.

Cas 1 – c'est une ligne :

`rowNb < gridHeight`

on compare la même ligne et la même colonne : `allPossibility[rowNb][i][j]` avec `grid[rowNb][j]`

Cas 2 : c'est une colonne :

`rowNb >= gridHeight`

On compare : `allPossibility[rowNb][i][j]` avec `grid[j][rowNb-gridHeight]`

`j` devient l'indice vertical et `rowNb-gridHeight` donne l'indice de colonne.

Suppression de la ligne invalide de la liste : `allPossibility[rowNb].splice(i,1)`

Résumé : Pour chaque possibilité : Si elle contredit une info connue de la grille : on la supprime !!!

`editGrid(rowNb)`

Cette fonction regarde toutes les possibilités restantes d'une ligne ou colonne, et si 100% d'entre elles ont la même valeur à une position donnée, alors on écrit cette valeur dans la grille.

`if (rowNb < gridHeight) length = gridWidth =>` on parcourt les lignes

`else length = gridHeight =>` on parcourt les colonnes

`for (let j = 0; j < length; j++) =>` pour chaque position `j`

`for (let i = 0; i < allPossibility[rowNb].length; i++) =>` on regarde toutes les possibilités restantes.

`if (i == 0)`

`comparaison = valeur`

`else if (valeur différente)`

`edit = false`

On prend la valeur de la première possibilité comme référence.

Si une seule possibilité diffère : On ne modifie rien.

Si toutes sont identiques : On peut écrire dans la grille.

Fonction `copy(grid)`

Créer une copie indépendante de la grille à copier. La copie de la grille est stockée dans une nouvelle liste identique res.

Fonction `compareGrid(grid1, grid2)`

Vérifier si deux grilles sont strictement identiques.

On compare chaque case des deux grilles

```
if(grid1[i][j] != grid2[i][j])
```

```
    return false
```

Fonction `printGrid(grid)`

Afficher la grille dans la console avec :

- les indices des colonnes au-dessus
- les indices des lignes à gauche

Étape 1 : calculer la hauteur maximale des indices colonnes

Trouver la ligne avec le plus d'indices. On compare ligne par ligne la longueur de la liste et on sauvegarde le plus long dans `longestRowhintList`.

Le variable `strEspaceHint` = ' ' sert pour préparer l'espace avant la grille. On commence avec 2 espaces (' ') de base.

```
for (let i = 0; i < longestRowhintList * 2; i++){
```

```
    strEspaceHint += ' ' }
```

puis on parcourt le `longestRowhintList * 2` pour ajouter autant d'espace qu'il y a d'indices. (*2 car on ajoute une espace après chaque valeur)

Cette partie est nécessaire pour que la grille commence toujours au même endroit.

Résultat avant :

```
2 | 1 1 0 1
```

```
1 1 1 | 1 0 1 1
```

Après :

```
2 | 1 1 0 1
```

```
1 1 1 | 1 0 1 1
```

Puis on cherche la colonne avec le plus d'indices. On compare colonne par colonne la longueur de la liste et on sauvegarde le plus long dans `longestColumnhintList`.

Si `longestColumnhintList = 3` => cela signifie qu'on devra afficher 3 lignes d'indices au-dessus de la grille.

Alignement vertical

```
if (longestColumnhintList - hintListColumns[j].length <= i)
```

cette condition permet d'aligner les indices par le bas => Les colonnes avec moins d'indices doivent commencer plus bas.

Puis on ajoute la ligne de séparation pour séparer les indices colonnes de la grille. :

```
splitHintGrid = "
```

```
for (let i = 0; i < gridWidth; i++){
```

```
    splitHintGrid += '- '
```

```
} console.log(strEspaceHint + splitHintGrid)
```

Fonction Solver()

C'est le moteur principal du solveur.

Étape 1 : Sauvegarde

`gridImage = copy(grid)` - On garde une copie pour voir si la grille change.

Étape 2 : Pour chaque ligne et colonne

```
for (i = 0; i < gridHeight + gridWidth; i++)
```

on appelle les fonctions `reducePossibilities(i)` et `editGrid(i)`

On supprime les possibilités impossibles et on écrit les certitudes.

Étape 3 : Vérifier si la grille a changé

```
if (compareGrid(grid, gridImage)) break
```

Si la grille n'a pas changé : Plus rien à déduire, on arrête

Étape 4 : Vérifier si résolu

```
return compareGrid(grid, gridSolution)
```

Retourne : true si la grille correspond à la solution

false sinon

`solve()` applique réduction + déduction en boucle jusqu'à ce qu'il n'y ait plus de changement et on arrête si la condition - `if (compareGrid(grid, gridImage))` est vrai.

Fonction finPartie()

Vérifie si la grille correspond exactement à la solution.

On parcourt toutes les cases et on vérifie:

```
if (  
    solution == 1 && grid != 1  
    ||  
    solution != 1 && grid == 1  
)  
    return false
```

Les cases remplies doivent correspondre exactement.

La boucle while ligne 278

Cette boucle est la boucle principale du programme.

Elle génère une grille solution aléatoire, crée les indices, prépare les possibilités, lance le solveur et recommence tant qu'une solution valide n'est pas trouvée.

Déroulé du jeu:

1 Création de gridSolution

→ Grille aléatoire remplie de 1 ou 2 (remplie ou marquée).

2 Création de grid

→ Grille vide (remplie de 0)

C'est celle que le solveur va essayer de compléter.

3 Création des colonnes

On transforme la solution en colonnes pour pouvoir calculer les indices colonnes.

4 Création des indices (grâce à la fonction createHintList())

hintListRows

hintListColumns

5 Génération de toutes les possibilités (grâce à la fonction allPossibilityList())

Pour :

- toutes les lignes
- toutes les colonnes

Ces possibilités sont ensuite stockées dans la liste allPossibility

6 Appel du solveur

if (solve()) break

Si le solveur réussit :

→ on sort de la boucle

Sinon :

→ on recommence avec une nouvelle grille aléatoire

La boucle while (ligne 349)

1 Boucle principale du jeu (pas le solveur comme la boucle d'avant).

On continue la boucle tant que finPartie() == false, donc :

Tant que la grille du joueur n'est pas identique à la solution.

Quand la grille est correcte → finPartie() retourne true → la boucle s'arrête.

2 Affichage de la grille

printGrid(grid)

On affiche la grille actuelle du joueur.

3 Boucle de saisie

```
while (inputLigne < 0 || inputColonne < 0)
```

Cela signifie :

Tant qu'une ligne ou colonne valide n'est pas entrée, on redemande.

On doit saisir d'abord le numéro de la ligne de 0 au 9 et puis le numéro de la colonne de 0 à 9 (si la taille de la grille est 10*10)

si la valeur saisie n'est pas valide:

- ce n'est pas un nombre
- ou le nombre est trop grand

→ on remet inputLigne à -1 pour forcer la boucle à recommencer et redemander au joueur de choisir une ligne/colonne.

4 Modification de la case :

```
grid[inputLigne][inputColonne] =
```

```
(grid[inputLigne][inputColonne] + 1) % 3
```

Ce code fait tourner la valeur de la case :

0 → 1

1 → 2

2 → 0

(valeur + 1) % 3

Donc chaque clic change l'état :

- 0 = vide
- 1 = rempli
- 2 = croix

5 Réinitialisation des inputs

on remet les valeurs à -1 :

```
inputLigne = -1
```

```
inputColonne = -1
```

pour forcer la prochaine saisie.

6 Fin de partie

Quand finPartie() devient true :

```
printGrid(grid)
```

```
printGrid(gridSolution)
```

```
console.log('GG')
```

On affiche :

- la grille finale
- la solution
- message de victoire